C++程序设计实验报告

> 实验题目 [类与对象（2）]
>
> 学 号 [__302025513022__]
>
> 姓 名 [__周林钥__]
>
> 班 级 [__健行理工数智二班__]

提交日期 [2026.3.23]

---

1. 实验目的：

1. 掌握构造函数、析构函数、复制构造函数的定义与使用
2. 掌握 `const` 成员函数与 `mutable` 成员变量的用法
3. 掌握静态数据成员与静态成员函数的定义与使用
4. 掌握友元函数的使用

---

2. 实验内容：

1. 设计 `DataInfo` 类，实现无参构造函数、有参构造函数、复制构造函数、析构函数、日期设置与获取功能，并编写主函数验证。
2. 完成教材 P279–280 第 6、7、8 题（`const` 相关内容）。
3. 完成教材 P280 第 9 题（静态成员），编写 `Sale` 类，计算销售总额与平均售价。
4. 完成教材 P281 第 10 题（友元），编写 `Time` 和 `Date` 类，用友元函数 `display` 输出时间与日期。

三、实验过程

任务一：DataInfo 类

（1）程序代码

DataInfo.h

```
#include <bits/stdc++.h>
using namespace std;

class DataInfo{
    private:
        int yy;
        int mm;
        int dd;
    public:
        DataInfo();  //无参数构造函数
        DataInfo(int, int, int);  //有参构造函数
        DataInfo(const DataInfo&);  //复制构造函数
        ~DataInfo();  //析构函数
        void set();  //实现日期设置
        void get();  //实现日期获取
```

```
};
```

DataInfo.cpp

```
#include "DataInfo.h"
using namespace std;

DataInfo::DataInfo(){
    yy = 2026;
    mm = 3;
    dd = 23;
    cout << "无参构造函数被调用" << endl;
}

DataInfo::DataInfo(int yy, int mm, int dd):yy(yy), mm(mm), dd(dd){
    cout << "有参构造函数被调用" << endl;
}

DataInfo::DataInfo(const DataInfo& tmp){
    yy = tmp.yy;
    mm = tmp.mm;
    dd = tmp.dd;
    cout << "复制构造函数被调用" << endl;
}

DataInfo::~DataInfo(){
    yy = 0;
    mm = 0;
    dd = 0;
    cout << "析构函数被调用" << endl;
}

void DataInfo::set(){
    int y, m, d;
    cout << "请依次输入年、月、日（空格隔开）" << endl;
    cin >> y >> m >> d;
    yy = y;
    mm = m;
    dd = d;
}

void DataInfo::get(){
    cout << "日期: " << yy << "年" << mm << "月" << dd << "日" << endl;
}
```

main.cpp

```
#include "DataInfo.h"
#include <windows.h>
using namespace std;

int main(){
```

```
    SetConsoleOutputCP(CP_UTF8);
    DataInfo DataInfo1;  //无参构造函数
    DataInfo DataInfo2(6666, 66, 66);  //有参构造函数
    DataInfo DataInfo3(DataInfo1);  //复制构造函数
    DataInfo1.set();
    DataInfo1.get();
    system("pause");
    return 0;
}
```

（2）运行结果

```
无参构造函数被调用
有参构造函数被调用
复制构造函数被调用
请依次输入年、月、日（空格隔开）
2026 3 23
日期：2026年3月23日
析构函数被调用
析构函数被调用
析构函数被调用
```

（3）结果分析

无参构造函数、有参构造函数、复制构造函数均按预期被调用，析构函数在对象生命周期结束时自动调用。`set()` 和 `get()` 成员函数实现了日期的设置与输出。

任务二：const 成员（第 6、7、8 题）

（1）第 6 题

将 `stud` 声明为常对象后，常对象不能调用非常成员函数，因此 `change()` 和 `display()` 均无法调用，编译出错。

（2）第 7 题

修改方案：

- 将 `num` 和 `score` 声明为 `mutable`，允许在常成员函数中修改
- 将 `change()` 和 `display()` 声明为 `const` 成员函数

修改后代码编译通过，运行结果为：

```
101 78.5
101 80.5
```

（3）第 8 题

将函数参数改为引用传递，避免对象复制，同时可直接修改原对象：

```
void fun(Student& tmp){
    tmp.display();
    tmp.change(101, 80.5);
    tmp.display();
}
```

运行结果：

```
101 78.5
101 80.5
```

（4）结果分析

通过 `const` 关键字可以限制成员函数的修改能力，`mutable` 提供了一种在常成员函数中修改特定成员的途径。引用传递避免了复制开销，且可直接修改原对象。

任务三：静态成员（第 9 题）

（1）程序代码

```
#include <bits/stdc++.h>
using namespace std;

class Sale{
    private:
        int num;
        int quantity;
        double price;

        static double discount;
        static double sum;   //总销售额
        static int n;        //总销售件数

    public:
        Sale(int n, int q, double p) : num(n), quantity(q), price(p) {}

        static void setDiscount(){
            cout << "请输入discount: " << endl;
            cin >> discount;
        }

        void calculate(){
            double salePrice = price;
            if (quantity > 10){
                salePrice *= 0.98;
            }
            salePrice *= discount;
            double saleMoney = salePrice * quantity;
            sum += saleMoney;
            n += quantity;
        }
```

```
        static double average(){
            if (n == 0) return 0;
            return sum / n;
        }

        static void display(){
            cout << fixed << setprecision(2);
            cout << "sum:" << sum << endl;
            cout << "average:" << average() << endl;
        }
};

//类外初始化静态数据成员
double Sale::discount = 0.95;
double Sale::sum = 0;
int Sale::n = 0;

int main(){
    Sale sales[3] = {
        {101, 5, 23.5},
        {102, 12, 24.56},
        {103, 100, 21.5}
    };
    Sale::setDiscount();
    for (int i = 0; i < 3; i++){
        sales[i].calculate();
    }
    Sale::display();
    return 0;
}
```

（2）运行结果

```
请输入discount:
0.95
sum:2624.57
average:21.65
```

（3）结果分析

静态成员 `discount`、`sum`、`n` 属于类本身，所有对象共享。静态成员函数 `setDiscount()`、`average()`、`display()` 可以直接通过类名调用，用于操作全局的销售统计数据。类外初始化是静态成员分配内存的必要步骤。

任务四：友元函数（第 10 题）

（1）程序代码

```
#include <bits/stdc++.h>
using namespace std;

class Date;
```

```cpp
class Time{
    public:
        Time(int h, int m, int s) : hour(h), minute(m), sec(s){}
        friend void display(Date&, Time&);
    private:
        int hour;
        int minute;
        int sec;
};

class Date{
    public:
        Date(int y, int m, int d) : year(y), month(m), day(d){}
        friend void display(Date&, Time&);
    private:
        int year;
        int month;
        int day;
};

void display(Date& D, Time& T){
    cout << D.year << "年" << D.month << "月" << D.day << "日" << endl;
    cout << T.hour << ":" << T.minute << ":" << T.sec << endl;
}

int main(){
    Time t1{10, 36, 56};
    Date d1{12, 25, 2024};
    display(d1, t1);
    return 0;
}
```

(2) 运行结果

```
2024年12月25日
10:36:56
```

(3) 结果分析

通过友元函数 `display`，可以访问 `Time` 和 `Date` 类的私有成员。友元函数不是类的成员函数，但享有访问私有成员的权限，适用于需要跨类操作且不宜将数据公开的场景。

四、小结与收获

通过本次实验，我对 C++ 类的核心机制有了更深入的理解：

- **构造函数与析构函数**：掌握了无参、有参、复制构造函数的定义时机与调用顺序，析构函数在对象销毁时自动执行，用于资源释放。
- **const 成员**：理解了常成员函数不能修改普通成员，但可通过 `mutable` 修饰允许特定成员被修改；引用传递避免了对象复制，提高了效率。

- **静态成员**：静态数据成员属于类本身，所有对象共享，必须在类外初始化；静态成员函数可直接通过类名调用，用于操作全局数据。
- **友元函数**：友元函数可以访问类的私有成员，适用于需要跨类访问的场景，但破坏了封装性，应谨慎使用。

【毕业要求3】设计/开发解决方案：本次实验中的各类设计，体现了面向对象编程中数据封装与功能模块化的思想，符合工程化开发的基本要求（支撑指标点3-3）。

【毕业要求5】使用现代工具：通过编写多文件项目、使用友元函数、静态成员等特性，熟悉了 C++ 标准库的使用和项目组织方式（支撑指标点5-1）。